T.C.

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BULUT BİLİŞİM DERSİ (2026-2027 GÜZ YARIYILI)

HAFTA 6 UYGULAMA RAPORU: KUBERNETES İLE KONTEYNER ORKESTRASYONU, MİKROSERVİS DAĞITIMI, KALICI DEPOLAMA, SAĞLIK KONTROLLERİ, ÖLÇEKLEME VE BULUTA GEÇİŞ DEĞERLENDİRMESİ

Öğrenci Adı Soyadı:	**Arda Kumartaşlıoğlu**
Öğrenci Numarası:	**16008122022**
Ders Adı:	Bulut Bilişim
Çalışma Haftası / Konusu:	Hafta 6 - Kubernetes ve Konteyner Orkestrasyonu
Teslim Tarihi:	Kasım 2026

1. GİRİŞ VE ÇALIŞMANIN AMACI

Modern bulut bilişim ekosisteminde konteynerleştirme teknolojileri (Docker vb.), uygulamaların bağımlılıklarıyla birlikte yalıtılmış ve taşınabilir bir biçimde paketlenmesini sağlamıştır. Ancak, yüzlerce mikroservisten oluşan ve dinamik yük altında çalışan büyük ölçekli sistemlerde münferit konteynerlerin manuel olarak yönetilmesi, düğüm arızalarının karşılanması, ağ yönlendirmesi, yük dengeleme ve veri sürekliliğinin temini insan kapasitesini aşmaktadır. İşte bu noktada konteyner orkestrasyonu devreye girmektedir.

Kubernetes, konteynerize edilmiş iş yüklerinin otomatik dağıtımı, ölçeklenmesi, yük dengelemesi ve kendi kendini iyileştirmesi (self-healing) için tasarlanmış açık kaynaklı, endüstri standardı bir orkestrasyon platformudur. Kubernetes mimarisinin merkezinde bildirimsel (declarative) yönetim ilkesi yer alır. Bu ilkeye göre sistem yöneticisi veya yazılım mühendisi 'ne yapılması gerektiğini' adım adım komutlarla emretmek (imperative yaklaşım) yerine; sistemin 'hangi nihai durumda bulunması gerektiğini' (desired state) YAML manifestoları aracılığıyla beyan eder. Kubernetes kontrol düzlemi (control plane) içerisindeki kontrol döngüleri (controller loops), kümenin anlık durumunu (actual state) sürekli gözlemleyerek bildirilen durum ile gerçek durum arasındaki farkı uzlaştırır (reconciliation).

Bu rapor; Hafta 6 kapsamında gerçekleştirilen yerel Kubernetes kümesi hazırlığını, 'Ders Notları Kayıt Uygulaması'nın mikroservis mimarisine uygun olarak Kubernetes nesneleriyle (Deployment, Service, ConfigMap, Secret, PersistentVolumeClaim) dağıtılmasını, veri kalıcılığının doğrulanmasını, kontrollü kesinti deneyleriyle sağlık kontrollerinin (readiness ve liveness probes) davranışlarının incelenmesini, elle yatay ölçeklemeyi, kademeli güncelleme (rolling update) ve geri dönüş (rollout undo) mekanizmalarını ve son olarak yönetilen bulut ortamlarına (AWS EKS, Azure AKS, Google Cloud GKE) geçiş dinamiklerini kapsamlı bir biçimde belgelemektedir.

> **[NOTE] Temel Hedef ve Kriterler**
> Bu çalışmanın başarı kriteri, Pod'ların yalnızca 'Running' durumunda görünmesi değildir. Uygulamanın uçtan uca çalışabilmesi, bileşenlerin iç ağ üzerinden iletişim kurabilmesi, kalıcı depolama sayesinde verilerin Pod silinmelerinden etkilenmemesi, sağlık denetimlerinin hata durumlarında trafiği doğru yönetmesi ve güncelleme/geri dönüş adımlarının veri kaybı olmaksızın gerçekleştirilmesi deneysel kanıtlarla doğrulanmıştır.

2. ÇALIŞMA ORTAMI VE YEREL KÜME HAZIRLIĞI

Kubernetes çalışmalarında yerel geliştirme ve test ortamları için Docker Desktop (kubeadm/kind) ve Minikube iki ana alternatif olarak öne çıkmaktadır. Bu çalışmada kılavuzun

(Ek 1) önerdiği ilkeler doğrultusunda, izole ve tekrarlanabilir bir küme ortamı sunan Minikube platformu tercih edilmiştir. Minikube üzerinde 'hafta6' adında özel bir profil tanımlanmış, Linux çekirdeğinin yerel KVM2 (Kernel-based Virtual Machine) hipervizör sürücüsü ve modern 'containerd (v2.3.4)' konteyner çalışma zamanı kullanılarak tek düğümlü bir Kubernetes v1.37.0 kümesi ayağa kaldırılmıştır.

2.1 Küme Bileşenleri ve Düğüm Durumu

Kümenin başlatılmasının ardından 'kubectl' komut satırı aracı küme bağlamına (context) yönlendirilmiş ve küme düğümünün sağlık durumu denetlenmiştir. 'kubectl get nodes' komutu ile kontrol düzlemi ve işçi rollerini üstlenen 'hafta6' düğümünün 'Ready' durumunda olduğu teyit edilmiştir:

Terminal / Kod: Düğüm ve Bağlam Doğrulaması

```
$ kubectl config current-context
hafta6

$ kubectl get nodes -o wide
NAME     STATUS   ROLES           AGE   VERSION   INTERNAL-IP       OS-IMAGE
KERNEL-VERSION     CONTAINER-RUNTIME
hafta6   Ready    control-plane   2m    v1.37.0   192.168.122.187   Buildroot 2023.02.9
6.6.152            containerd://2.3.4
```

2.2 Depolama Sınıfı (StorageClass) ve Ad Alanı (Namespace) Hazırlığı

Kalıcı verilerin (stateful data) yönetilebilmesi için kümede dinamik depolama sağlayıcısının bulunması zorunludur. Minikube ortamında varsayılan olarak gelen 'standard (k8s.io/minikube-hostpath)' depolama sınıfı kontrol edilmiştir. Bu depolama sınıfı, kalıcı birim talepleri (PVC) oluşturulduğunda ana makinedeki bir dizini otomatik olarak PersistentVolume (PV) olarak bağlamaktadır. Ancak varsayılan Reclaim Policy (Geri Kazanım Politikası) değerinin 'Delete' olduğuna dikkat edilmelidir.

Farklı projelerin ve sistem kaynaklarının birbirini etkilememesi için kılavuz gereği 'hafta6' adında bağımsız bir ad alanı (Namespace) oluşturulmuş ve tüm nesneler bu mantıksal sınır içerisinde konumlandırılmıştır:

Terminal / Kod: StorageClass ve Namespace Doğrulaması

```
$ kubectl get storageclass
NAME                 PROVISIONER                RECLAIMPOLICY   VOLUMEBINDINGMODE
ALLOWVOLUMEEXPANSION   AGE
standard (default)   k8s.io/minikube-hostpath   Delete          Immediate
false                  2m
```

```
$ kubectl create namespace hafta6
namespace/hafta6 created

$ kubectl get namespace hafta6
NAME     STATUS   AGE
hafta6   Active   15s
```

3. UYGULAMA MİMARİSİ VE KUBERNETES NESNELERİ

Ders kapsamında kullanılan 'Ders Notları Kayıt Uygulaması', iki ana katmandan oluşan tipik bir mikroservis mimarisidir:

- **Web Uygulama Katmanı:** Kullanıcı isteklerini karşılayan, web formunu sunan, notları listeleyen ve REST sağlık uç noktalarını (/health/ready, /health/live) barındıran Flask tabanlı web servisi.
- **Veritabanı Katmanı:** Ders notlarını ilişkisel tabloda (dersnotlari) kalıcı olarak saklayan, ACID güvencesine sahip PostgreSQL 16 veritabanı servisi.

Aşağıdaki tabloda, bu iki katmanın Kubernetes üzerinde bildirimsel olarak yönetilmesi için tanımlanan temel nesneler ve görevleri özetlenmiştir:

Kubernetes Nesnesi	Nesne Adı	Türü / Stratejisi	Görevi ve Sağladığı Fonksiyon
ConfigMap	app-config	Yapılandırma	Hassas olmayan DB_HOST, DB_PORT, DB_NAME, DB_USER ve COOKIE_SECURE ayarlarını tutar.
Secret	app-secrets	Opaque (Gizli Bilgi)	Veritabanı parolası (db_password) ve Flask oturum anahtarını (app_secret) salt-okunur dosyalar olarak saklar.
PersistentVolumeClaim	db-data	1Gi / ReadWriteOnce	PostgreSQL için varsayılan StorageClass üzerinden kalıcı disk alanı tahsis eder.
Deployment (DB)	db	Recreate (1 kopya)	PostgreSQL 16 motorunu çalıştırır, aynı

			anda iki Pod'un diske yazmasını engelleyen Recreate stratejisini kullanır.
Deployment (Web)	web	RollingUpdate (1-2 kopya)	Web uygulamasını çalıştırır, sıfır kesintili aşamalı güncelleme (maxUnavailable: 0, maxSurge: 1) uygular.
Service (DB)	db	ClusterIP (Port 5432)	Veritabanını küme içi DNS'e açar (db:5432), internete doğrudan maruz kalmasını engeller.
Service (Web)	web	ClusterIP (Port 8000)	Web Pod'larına yük dengeli erişim sunar ve EndpointSlice nesneleriyle sağlıklı hedefleri yönetir.
Pod (Kontrol)	check	Standalone Pod	Küme içerisinden DNS çözümlemesini ve iç servis iletişimini doğrulamak için kullanılan test Pod'u.

3.1 Güvenlik ve Gizli Bilgilerin Ayrımı (Secret vs ConfigMap)

On İki Faktörlü Uygulama (The Twelve-Factor App) metodolojisi uyarınca, yapılandırma ve kimlik bilgileri kaynak koddan kesinlikle ayrılmalıdır. Bu ilke gereğince gizli olmayan ortam değişkenleri 'ConfigMap' nesnesinde saklanırken; veritabanı erişim şifresi ve kullanıcı oturumlarını imzalayan 'APP_SECRET' anahtarı 'Secret' nesnesi üzerinden sisteme dahil edilmiştir.

> **[WARN] Güvenlik Uyarısı: Base64 Şifreleme Değildir**
> Kubernetes Secret nesnelerinin YAML çıktısında (kubectl get secret -o yaml) değerler Base64 formatında kodlanmış olarak görünür. Base64 bir şifreleme (encryption) algoritması değil, yalnızca ikili veriyi metne dönüştüren bir kodlama (encoding) yöntemidir. Herhangi biri bu metni saniyeler içinde çözebilir. Bu nedenle Secret nesnelerinin içeriği asla doğrudan raporlara veya ekran görüntülerine eklenmemelidir. Çalışmamızda şifreler ortam değişkeni yerine Pod

> içine '/run/secrets/' dizinine salt-okunur (readOnly: true) volume olarak bağlanmış; böylece işlem dökümlerinde (process dumps) veya çevre değişkeni loglarında görünmesi engellenmiştir.

Terminal / Kod: Secret Nesnesinin Dosyalardan Oluşturulması

```
$ kubectl create secret generic app-secrets -n hafta6 \
    --from-file=db_password=./secrets/db_password.txt \
    --from-file=app_secret=./secrets/app_secret.txt
secret/app-secrets created

$ kubectl get secret app-secrets -n hafta6
NAME          TYPE     DATA   AGE
app-secrets   Opaque   2      15s
```

4. KONTROLLÜ DENEYLER VE DOĞRULAMA KANITLARI

Kılavuzda (Ek 2) tanımlanan tüm deneyler adım adım uygulanmış, terminal çıktıları ve nesne durumları anlık olarak kayıt altına alınmıştır. Her deneyin öncesi, deney esnası ve deney sonrası durumları aşağıda ayrıntılı teknik değerlendirmelerle sunulmuştur.

4.1 Deney 1: Uygulamanın Dağıtılması ve İlk Doğrulama (Ek 2 Bölüm 7)

Amaç: Bildirimsel YAML manifestolarının sözdizimi doğruluğunu sınamak (dry-run), nesneleri hafta6 ad alanında oluşturmak, Pod'ların READY durumuna geçtiğini teyit etmek, port yönlendirme ile dış erişim sağlamak ve ilk kalıcılık test kayıtlarını oluşturmak.

Uygulanan Komutlar ve Süreç: Önce 'kubectl apply --dry-run=client -f ./k8s' ile istemci taraflı doğrulama yapılmış, ardından sırasıyla ConfigMap, Secret, PVC, DB Deployment, Servisler ve Web Deployment kümeye uygulanmıştır. Rollout kontrolleriyle her iki dağıtımın tamamlandığı izlenmiştir:

Terminal / Kod: Deney 1 Dağıtım Çıktıları

```
$ kubectl apply --dry-run=client -f ./k8s
configmap/app-config configured (dry run)
persistentvolumeclaim/db-data configured (dry run)
deployment.apps/db created (dry run)
service/db created (dry run)
service/web created (dry run)
deployment.apps/web created (dry run)

$ kubectl apply -f ./k8s/01-config.yaml
```

```
$ kubectl apply -f ./k8s/02-pvc.yaml
$ kubectl apply -f ./k8s/03-db.yaml
$ kubectl apply -f ./k8s/04-services.yaml
$ kubectl apply -f ./k8s/05-web.yaml

$ kubectl rollout status deployment/db -n hafta6 --timeout=180s
deployment "db" successfully rolled out

$ kubectl rollout status deployment/web -n hafta6 --timeout=180s
deployment "web" successfully rolled out

$ kubectl get deployments,pods,services,pvc -n hafta6
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/db    1/1     1            1           12s
deployment.apps/web   1/1     1            1           11s

NAME                         READY   STATUS    RESTARTS   AGE
pod/db-77cfdc4ccc-zm7rl    1/1     Running   0          12s
pod/web-58c776859c-785d7   1/1     Running   0          11s

NAME          TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)    AGE
service/db    ClusterIP   10.111.73.51   <none>        5432/TCP   11s
service/web   ClusterIP   10.109.84.91   <none>        8000/TCP   11s

NAME                             STATUS   VOLUME
CAPACITY   ACCESS MODES   STORAGECLASS   AGE
persistentvolumeclaim/db-data   Bound    pvc-fff7d32f-6b0a-478c-9943-f274a6f5ed86   1Gi
RWO            standard       17s
```

Dış Erişim ve İlk Test Kayıtları: Web servisi 'ClusterIP' türünde olduğundan küme dışından doğrudan IP ile ulaşılamaz. Geliştirme ve doğrulama amacıyla geçici port yönlendirme (port-forward) başlatılmıştır:
`kubectl port-forward -n hafta6 service/web 8080:8000 --address 127.0.0.1`
Ardından 'http://localhost:8080' üzerinden sağlık kontrolü yapılmış ve ilk iki test kaydı (Kalıcılık A ve Kalıcılık B) oluşturulmuştur:

Terminal / Kod: Sağlık Sorgusu ve İlk Kayıtların Eklenmesi

```
$ curl -s http://127.0.0.1:8080/health/ready
{"status":"ready","version":"1.1"}

$ curl -s -X POST http://127.0.0.1:8080/ -d "numara=101&baslik=Kalıcılık A&aciklama=İlk
kalıcılık test kaydı"
$ curl -s -X POST http://127.0.0.1:8080/ -d "numara=102&baslik=Kalıcılık
B&aciklama=İkinci kalıcılık test kaydı"
```

```
$ curl -s http://127.0.0.1:8080/ | grep -A 2 "Kalıcılık"
<td><strong>101</strong></td>
<td>Kalıcılık A</td>
<td>İlk kalıcılık test kaydı</td>
--
<td><strong>102</strong></td>
<td>Kalıcılık B</td>
<td>İkinci kalıcılık test kaydı</td>
```

Gözlemlenen Sonuç ve Değerlendirme: Hem veritabanı hem de web bileşenleri 1/1 hazır (READY) duruma geçmiş; kalıcı birim talebi (db-data) başarıyla 'Bound' statüsüne ulaşmıştır. Web uygulaması /health/ready sorgusuna 200 OK ile 'status=ready, version=1.1' yanıtı vermiş, veritabanı bağlantısının ve yazma kabiliyetinin eksiksiz çalıştığı doğrulanmıştır.

4.2 Deney 2: Service Üzerinden Küme İçi Ağ İletişimi Doğrulaması (Ek 2 Bölüm 8)

Amaç: Port yönlendirme bağımsız olarak, Kubernetes iç ağında DNS tabanlı servis keşfini (Service Discovery) ve EndpointSlice üzerinden yük yönlendirmesini doğrulamak.

Uygulanan Komutlar ve Süreç: Aynı kümede geçici bir 'check' Pod'u ('k8s/06-check.yaml') ayağa kaldırılmış ve bu Pod içerisinden web servisine 'http://web:8000/health/ready' adresi üzerinden istek gönderilmiştir. Ayrıca web servisinin arkasındaki Pod hedeflerini tutan EndpointSlice nesnesi sorgulanmıştır:

Terminal / Kod: Deney 2 Küme İçi DNS ve EndpointSlice Çıktısı

```
$ kubectl apply -f ./k8s/06-check.yaml
pod/check created
$ kubectl wait -n hafta6 --for=condition=Ready pod/check --timeout=120s
pod/check condition met

$ kubectl exec -n hafta6 check -- python -c "import urllib.request;
print(urllib.request.urlopen('http://web:8000/health/ready',
timeout=10).read().decode())"
{"status":"ready","version":"1.1"}

$ kubectl get endpointslices -n hafta6 -l kubernetes.io/service-name=web -o yaml
apiVersion: discovery.k8s.io/v1
kind: EndpointSlice
metadata:
  labels:
    kubernetes.io/service-name: web
  name: web-88twv
  namespace: hafta6
```

```
endpoints:
- addresses:
  - 10.244.0.4
  conditions:
    ready: true
    serving: true
    terminating: false
  nodeName: hafta6
  targetRef:
    kind: Pod
    name: web-58c776859c-785d7
    namespace: hafta6
    uid: a8e9f175-c9bc-4433-8692-72cd35184073
ports:
- port: 8000
  protocol: TCP
```

Gözlemlenen Sonuç ve Değerlendirme: 'check' Pod'u içerisindeki CoreDNS çözücüsü 'web' ismini 'service/web'in sanal ClusterIP adresine (10.109.84.91) başarıyla çözümlemiş ve kube-proxy bu trafiği EndpointSlice'ta 'ready: true' olarak işaretlenmiş olan 'web-58c776859c-785d7' (IP: 10.244.0.4) Pod'una iletmiştir. Böylece küme içi mikroservis iletişiminin standart Kubernetes mekanizmalarıyla sorunsuz işlediği ispatlanmıştır.

4.3 Deney 3: Veri Kalıcılığı ve Pod Yenilenme Doğrulaması (Ek 2 Bölüm 9.1)

Amaç: PostgreSQL veritabanı Pod'u silindiğinde veya arızalandığında, verilerin konteynerin geçici (ephemeral) katmanında değil, PersistentVolume (PV) üzerinde korunduğunu; Deployment denetleyicisinin yeni bir Pod oluşturup aynı birimi bağlayarak verileri eksiksiz geri sunduğunu kanıtlamak.

Uygulanan Komutlar ve Süreç: Mevcut veritabanı Pod'unun adı ve benzersiz sistem kimliği (UID) kaydedilmiş, ardından Pod zorla silinmiştir (kubectl delete pod -n hafta6 -l app=db). Deployment'ın yeni Pod'u oluşturması beklenmiş, yeni Pod'un UID'si alınmış ve web arayüzünden kayıtların durumu kontrol edilmiştir:

Terminal / Kod: Deney 3 Veritabanı Pod Silinmesi ve Veri Kalıcılığı Çıktısı

```
# 1. Silme Öncesi Durum
Eski DB Pod: db-77cfdc4ccc-zm7rl (UID: 2e0b5de3-faa6-43fe-ba12-ac6942b7c8a3)
PVC Durumu: db-data | Bound | pvc-fff7d32f-6b0a-478c-9943-f274a6f5ed86 | 1Gi | RWO

# 2. DB Pod'unun Silinmesi ve Yeniden Dağıtılması
$ kubectl delete pod -n hafta6 -l app=db
```

```
pod "db-77cfdc4ccc-zm7rl" deleted
$ kubectl rollout status deployment/db -n hafta6 --timeout=180s
deployment "db" successfully rolled out

# 3. Silme Sonrası Durum
Yeni DB Pod: db-77cfdc4ccc-5xcdx (UID: 5ded3e36-7e4d-4007-8614-ba6c4275e779)
PVC Durumu: db-data | Bound | pvc-fff7d32f-6b0a-478c-9943-f274a6f5ed86 | 1Gi | RWO

# 4. Web Arayüzünden Veri Kontrolü
$ curl -s http://127.0.0.1:8080/ | grep -A 2 "Kalıcılık"
<td><strong>101</strong></td>
<td>Kalıcılık A</td>
<td>İlk kalıcılık test kaydı</td>
--
<td><strong>102</strong></td>
<td>Kalıcılık B</td>
<td>İkinci kalıcılık test kaydı</td>
```

Gözlemlenen Sonuç ve Değerlendirme: Eski Pod (UID: 2e0b5de3...) silinmiş ve yerine tamamen yeni bir Pod (UID: 5ded3e36...) oluşturulmuştur. Ancak 'db-data' PVC nesnesi ve arkasındaki 'pvc-fff7d32f...' PV nesnesi hiç değişmeden sabit kalmıştır. Yeni veritabanı Pod'u ayağa kalktığında bu kalıcı diski '/var/lib/postgresql/data' altına bağlamış ve daha önce eklenmiş olan 101 (Kalıcılık A) ve 102 (Kalıcılık B) kayıtları sıfır veri kaybıyla aynen okunmuştur.

4.4 Deney 4: Web Pod'unun Silinmesi ve Self-Healing (Ek 2 Bölüm 9.2)

Amaç: Durumsuz (stateless) web bileşeninin bir Pod'u çöktüğünde veya silindiğinde, Deployment/ReplicaSet kontrol döngüsünün istenen kopya sayısını (replicas: 1) korumak için otomatik olarak yeni bir Pod üretmesini (Self-Healing) gözlemlemek.

Uygulanan Komutlar ve Süreç: Web Pod'unun UID'si alınmış ve 'kubectl delete pod -n hafta6 -l app=web' komutuyla silinmiştir:

Terminal / Kod: Deney 4 Web Pod Yenilenmesi Çıktısı

```
Eski Web Pod: web-58c776859c-785d7 (UID: a8e9f175-c9bc-4433-8692-72cd35184073)
$ kubectl delete pod -n hafta6 -l app=web
pod "web-58c776859c-785d7" deleted
$ kubectl rollout status deployment/web -n hafta6 --timeout=180s
deployment "web" successfully rolled out

Yeni Web Pod: web-58c776859c-66fwm (UID: 72b9967c-ffad-4010-9a70-acb8b8ee54a8)
$ curl -s http://127.0.0.1:8080/health/ready
{"status":"ready","version":"1.1"}
```

Gözlemlenen Sonuç ve Değerlendirme: Web Pod'unun silinmesiyle kümedeki hazır web kopya sayısı sıfıra düşmüş; ReplicaSet denetleyicisi bu farkı tespit ederek milisaniyeler içinde 'web-58c776859c-66fwm' adında yeni bir Pod başlatmıştır. Yeni Pod yapılandırmasını ConfigMap ve Secret'tan okuyarak hazır duruma geçmiş, port yönlendirme oturumu yenilendiğinde uygulamanın kesintisiz hizmet verdiği görülmüştür.

4.5 Deney 5: Sağlık Kontrolleri 1 - Veritabanı Kesintisi ve Readiness Probe (Ek 2 Bölüm 10.1)

Amaç: Bağımlı bir serviste (PostgreSQL) kesinti yaşandığında, Hazır Olma Kontrolünün (Readiness Probe) ve Canlılık Kontrolünün (Liveness Probe) nasıl farklı davrandığını ve Kubernetes servis trafiğinin nasıl yönetildiğini deneysel olarak kanıtlamak.

Uygulanan Komutlar ve Süreç: Veritabanı dağıtımı 'kubectl scale deployment/db -n hafta6 --replicas=0' komutuyla tamamen durdurulmuştur. Web Pod'unun ve EndpointSlice'ın durumu incelenmiş; ayrıca küme içi 'check' Pod'undan istek gönderilmiştir. Ardından db kopyası tekrar 1'e çıkarılarak toparlanma süreci izlenmiştir:

Terminal / Kod: Deney 5 Veritabanı Kesintisi ve Readiness Probe Çıktısı

```
# 1. DB Kopyası 0'a İndirildi (Yapay Kesinti)
$ kubectl scale deployment/db -n hafta6 --replicas=0
deployment.apps/db scaled

# 2. Kesinti Sırasında Web Pod Durumu ve EndpointSlice
$ kubectl get pods -n hafta6
NAME                   READY   STATUS    RESTARTS   AGE
check                  1/1     Running   0          93s
web-58c776859c-66fwm   0/1     Running   0          68s

$ kubectl get endpointslices -n hafta6 -l kubernetes.io/service-name=web -o yaml | grep
-A 4 "conditions:"
    conditions:
      ready: false
      serving: false
      terminating: false
    nodeName: hafta6

# 3. Canlılık (Liveness) vs Servis Trafiği (Readiness) Karşılaştırması
# İçeriden Liveness sorgusu: (Süreç ayakta, liveness bozulmadı)
$ kubectl exec -n hafta6 deployment/web -- python -c "import urllib.request;
print(urllib.request.urlopen('http://127.0.0.1:8000/health/live').read().decode())"
{"status":"ok"}

# Check Pod'undan Servis Üzerinden İstek: (Trafik kesildi!)
$ kubectl exec -n hafta6 check -- python -c "import urllib.request;
```

```
urllib.request.urlopen('http://web:8000/health/ready', timeout=5)"
Expected error during outage: <urlopen error [Errno 111] Connection refused>

# 4. DB'nin Tekrar Başlatılması ve Toparlanma
$ kubectl scale deployment/db -n hafta6 --replicas=1
deployment.apps/db successfully rolled out
pod/web-58c776859c-66fwm condition met

$ kubectl get pods -n hafta6
NAME                   READY   STATUS    RESTARTS   AGE
check                  1/1     Running   0          106s
db-77cfdc4ccc-5gfd5    1/1     Running   0          11s
web-58c776859c-66fwm   1/1     Running   0          81s

$ kubectl get endpointslices -n hafta6 -l kubernetes.io/service-name=web -o yaml | grep
-A 4 "conditions:"
    conditions:
      ready: true
      serving: true
      terminating: false
```

Gözlemlenen Sonuç ve Değerlendirme: Bu deney sağlık kontrollerinin en kritik ayrımını ortaya koymuştur:
1. Web Pod'unun konteyneri çökmemiştir (STATUS: Running), bu nedenle Liveness Probe başarılı olmaya devam etmiş ve kubelet Pod'u gereksiz yere yeniden başlatmamıştır (RESTARTS = 0).
2. Ancak web Pod'u veritabanına erişemediği için /health/ready uç noktası başarısız olmuş (HTTP 503) ve Pod 'READY: 0/1' durumuna düşmüştür.
3. EndpointSlice denetleyicisi, hazır olmayan bu Pod'u anında 'ready: false' olarak işaretlemiş ve Service üzerinden gelen kullanıcı trafiğini kesmiştir. Bu sayede kullanıcılar bozuk yanıtlar almak yerine trafikten izole edilmiştir.
4. Veritabanı tekrar açıldığında Readiness Probe başarı sağlamış, hedef otomatik olarak tekrar 'ready: true' yapılmış ve trafik kullanıcı müdahalesi gerekmeksizin yeniden akmaya başlamıştır. Ardından eklenen 'Kalıcılık C' kaydı sistemin tamamen toparlandığını doğrulamıştır.

4.6 Deney 6: Sağlık Kontrolleri 2 - Canlılık (Liveness Probe) Başarısızlığı (Ek 2 Bölüm 10.2)

Amaç: Uygulama kilitlendiğinde veya içsel bir çıkmaza girdiğinde (deadlock), Canlılık Kontrolünün (Liveness Probe) bu durumu nasıl tespit ettiğini ve kubelet'in arızalı konteyneri yeniden başlatarak (restart) sistemi nasıl kurtardığını gözlemlemek.

Uygulanan Komutlar ve Süreç: Web dağıtımında livenessProbe tanımına kontrollü bir test koşulu eklenmiştir: konteyner içinde '/tmp/live-fail' dosyası varsa kontrol 'assert not exists' ifadesiyle kasıtlı olarak başarısız olmaktadır. Pod UID'si kaydedilmiş, ardından dosya oluşturulup kubelet'in tepkisi izlenmiştir:

Terminal / Kod: Deney 6 Canlılık Kontrolü Başarısızlığı Çıktısı

```
# 1. Başlangıç Durumu
Web Pod: web-58c776859c-66fwm (UID: 72b9967c-ffad-4010-9a70-acb8b8ee54a8, RESTARTS: 0)

# 2. Hata Tetikleme Dosyasının Oluşturulması
$ kubectl exec -n hafta6 web-58c776859c-66fwm -- python -c "from pathlib import Path;
Path('/tmp/live-fail').touch()"

# 3. Kubelet Olayları ve Konteynerin Yeniden Başlatılması
$ kubectl describe pod web-58c776859c-66fwm -n hafta6
Events:
  Type     Reason     Age                From     Message
  ----     ------     ----               ----     -------
  Warning  Unhealthy  42s (x3 over 52s)  kubelet  Liveness probe failed: AssertionError
  Normal   Killing    42s                kubelet  Container web failed liveness probe,
will be restarted
  Normal   Pulled     12s                kubelet  Container image "dersnotlari-web:1.1"
already present on machine
  Normal   Created    12s                kubelet  Container created
  Normal   Started    12s                kubelet  Container started

# 4. Yeniden Başlatma Sonrası Durum
$ kubectl get pod web-58c776859c-66fwm -n hafta6
NAME                   READY   STATUS    RESTARTS      AGE
web-58c776859c-66fwm   1/1     Running   1 (11s ago)   2m37s

# 5. Pod UID Kontrolü ve Geçici Dosya Varlık Denetimi
Pod UID Değişti mi?: 72b9967c-ffad-4010-9a70-acb8b8ee54a8 (AYNI KALDI!)
$ kubectl exec -n hafta6 deployment/web -- python -c "from pathlib import Path;
print('live-fail exists?:', Path('/tmp/live-fail').exists())"
live-fail exists?: False
```

Gözlemlenen Sonuç ve Değerlendirme: Liveness Probe ardışık 3 kez (failureThreshold: 3) başarısız olunca kubelet doğrudan müdahale etmiş ve 'Container web failed liveness probe, will be restarted' olayıyla konteyneri sonlandırıp yeniden başlatmıştır. Burada can alıcı gözlem şudur: Pod'un kendisi silinmemiştir; Pod UID'si (72b9967c...) tamamen aynı kalmış, yalnızca Pod içindeki web konteyneri yeniden başlatılmış ve RESTARTS değeri 0'dan 1'e yükselmiştir. Konteyner yeniden başlatıldığında geçici yazılabilir katman sıfırlandığı için '/tmp/live-fail'

dosyası kendiliğinden yok olmuş ve uygulama kalıcı bir arıza döngüsüne girmeden toparlanmıştır.

4.7 Deney 7: Kaynak Talepleri ve Elle Yatay Ölçekleme (Ek 2 Bölüm 11)

Amaç: Konteyner kaynak yönetiminde 'requests' (garanti edilen kaynak) ve 'limits' (tavan sınır) kavramlarını doğrulamak; web bileşenini 1 kopyadan 2 kopyaya çıkararak EndpointSlice üzerindeki yük dağıtımını gözlemlemek ve veritabanının neden çoğaltılmadığını açıklamak.

Uygulanan Komutlar ve Süreç: Dağıtım tanımındaki kaynak sınırları ('limits: cpu 500m, memory 256Mi', 'requests: cpu 100m, memory 128Mi') kontrol edilmiş; ardından 'kubectl scale deployment/web -n hafta6 --replicas=2' çalıştırılarak 2 kopya hedeflenmiştir:

Terminal / Kod: Deney 7 Ölçekleme Çıktıları

```
$ kubectl describe deployment web -n hafta6 | grep -A 4 "Limits:"
    Limits:
      cpu:     500m
      memory:  256Mi
    Requests:
      cpu:     100m
      memory:  128Mi

# 1 -> 2 Kopya Ölçekleme
$ kubectl scale deployment/web -n hafta6 --replicas=2
deployment.apps/web scaled
$ kubectl rollout status deployment/web -n hafta6 --timeout=180s
deployment "web" successfully rolled out

$ kubectl get deployment web -n hafta6
NAME   READY   UP-TO-DATE   AVAILABLE   AGE
web    2/2     2            2           4m37s

$ kubectl get pods -n hafta6 -l app=web -o wide
NAME                   READY   STATUS    RESTARTS   AGE    IP           NODE
web-58c776859c-66fwm   1/1     Running   1          3m8s   10.244.0.7   hafta6
web-58c776859c-hsrrj   1/1     Running   0          6s     10.244.0.9   hafta6

$ kubectl get endpointslices -n hafta6 -l kubernetes.io/service-name=web -o yaml | grep
-A 4 "addresses:"
  - addresses:
    - 10.244.0.7
    conditions:
      ready: true
--
  - addresses:
    - 10.244.0.9
    conditions:
```

```
      ready: true

# 2 -> 1 Kopya İndirme
$ kubectl scale deployment/web -n hafta6 --replicas=1
deployment.apps/web scaled
$ kubectl get pods -n hafta6 -l app=web
NAME                   READY   STATUS        RESTARTS   AGE
web-58c776859c-66fwm   1/1     Running       1          3m8s
web-58c776859c-hsrrj   1/1     Terminating   0          6s
```

Gözlemlenen Sonuç ve Değerlendirme: Web kopya sayısı 2'ye çıkarıldığında Kubernetes Scheduler, kümedeki kaynak kapasitesini kontrol etmiş (toplam 200m CPU ve 256Mi RAM talebi) ve ikinci Pod'u ('web-...-hsrrj') düğüme başarıyla yerleştirmiştir. EndpointSlice nesnesi otomatik güncellenerek her iki Pod IP'sini (10.244.0.7 ve 10.244.0.9) 'ready: true' olarak listelemiş; böylece gelen istekler round-robin mantığıyla iki Pod arasında paylaştırılmıştır. Kopya sayısı tekrar 1'e indirildiğinde ikinci Pod kontrollü biçimde (Terminating) kaldırılmıştır.

> **[WARN] PostgreSQL Neden Aynı Şekilde Çoğaltılmaz?**
> Web bileşeni durumsuz (stateless) olduğu için kopya sayısı serbestçe artırılabilir; her Pod aynı veritabanına bağlanır ve gelen isteği bağımsızca işler. Buna karşın PostgreSQL ilişkisel ve durumsal (stateful) bir sistemdir. Kılavuzdaki kalıcı depolama (ReadWriteOnce) tek bir dosya sistemini bağlar. Birden fazla PostgreSQL Pod'u aynı veri dizinine (PGDATA) eşzamanlı yazmaya kalkışırsa 'split-brain' (bölünmüş beyin) durumu oluşur, işlem kilitleri bozulur ve veritabanı dosyaları geri dönülemez biçimde bozulur (data corruption). Veritabanını çoğaltmak için basit 'scale' komutu yetersizdir; birincil-ikincil (primary-replica) replikasyon protokolleri, Write-Ahead Logging (WAL) akışı ve StatefulSet gibi özelleşmiş orkestrasyon mimarileri gereklidir.

4.8 Deney 8: Kademeli Güncelleme (Rolling Update) ve Geri Dönüş (Rollout Undo) (Ek 2 Bölüm 12-13)

Amaç: Uygulamanın yeni sürümünün (1.2) sıfır kesinti stratejisiyle (maxUnavailable: 0, maxSurge: 1) nasıl dağıtıldığını gözlemlemek, yeni sürüm üzerinde veritabanına yeni kayıt eklemek, ardından 'rollout undo' ile önceki sürüme (1.1) geri dönüldüğünde hem eski hem yeni kayıtların korunduğunu kanıtlamak.

Uygulanan Komutlar ve Süreç: Web imajı 'dersnotlari-web:1.2' olarak güncellenmiş ve aşamalı geçiş izlenmiştir. Sürüm 1.2 hazır olunca '1.2 Sürüm Kaydı' başlıklı yeni bir not eklenmiştir. Ardından 'kubectl rollout undo deployment/web -n hafta6' komutu verilerek revizyon 1.1'e geri dönülmüş ve veritabanı kayıtları listelenmiştir:

Terminal / Kod: Deney 8 Rolling Update ve Rollout Undo Çıktıları

```
# 1. Sürüm 1.2'ye Güncelleme
$ kubectl set image deployment/web web=dersnotlari-web:1.2 -n hafta6
deployment.apps/web image updated
$ kubectl rollout status deployment/web -n hafta6 --timeout=180s
Waiting for deployment "web" rollout to finish: 0 out of 1 new replicas have been
updated...
Waiting for deployment "web" rollout to finish: 1 old replicas are pending
termination...
deployment "web" successfully rolled out

$ kubectl get deployment web -n hafta6 -o
jsonpath='{.spec.template.spec.containers[0].image}'
dersnotlari-web:1.2

# 2. Sürüm 1.2'de Sağlık ve Yeni Kayıt Ekleme
$ kubectl exec -n hafta6 check -- python -c "import urllib.request;
print(urllib.request.urlopen('http://web:8000/health/ready').read().decode())"
{"status":"ready","version":"1.2"}

# 1.2 sürümünde yeni kayıt (No: 104) eklenmesi:
<td>Kalıcılık A</td>
<td>Kalıcılık B</td>
<td>Kalıcılık C</td>
<td>1.2 Sürüm Kaydı</td>

# 3. Önceki Sürüme (1.1) Geri Dönüş (Rollout Undo)
$ kubectl rollout undo deployment/web -n hafta6
deployment.apps/web rolled back
$ kubectl rollout status deployment/web -n hafta6 --timeout=180s
deployment "web" successfully rolled out

$ kubectl get deployment web -n hafta6 -o
jsonpath='{.spec.template.spec.containers[0].image}'
dersnotlari-web:1.1

# 4. Geri Dönüş Sonrası Sağlık ve Veritabanı Bütünlüğü Kontrolü
$ kubectl exec -n hafta6 check -- python -c "import urllib.request;
print(urllib.request.urlopen('http://web:8000/health/ready').read().decode())"
{"status":"ready","version":"1.1"}

$ kubectl exec -n hafta6 check -- python -c "import urllib.request;
print(urllib.request.urlopen('http://web:8000/').read().decode())" | grep "<td>"
<td><strong>101</strong></td> <td>Kalıcılık A</td>
<td><strong>102</strong></td> <td>Kalıcılık B</td>
<td><strong>103</strong></td> <td>Kalıcılık C</td>
<td><strong>104</strong></td> <td>1.2 Sürüm Kaydı</td>
```

Gözlemlenen Sonuç ve Değerlendirme: RollingUpdate stratejisi 'maxUnavailable: 0' kuralına sadık kalarak, yeni sürüm 1.2 Pod'u tamamen başlatılıp hazır olma testini (readiness probe) geçmeden eski 1.1 Pod'unu sonlandırmamıştır. Böylece kullanıcılara kesintisiz hizmet sunulmuştur. Geri dönüş ('rollout undo') yapıldığında ise imaj tekrar 'dersnotlari-web:1.1'e geçmiş ve /health/ready 'version: 1.1' döndürmüştür. En kritik sonuç şudur: Uygulama sürümünün 1.1'e dönmesi, 1.2 sürümündeyken veritabanına eklenen '1.2 Sürüm Kaydı'nı kesinlikle silmemiştir! Çünkü Kubernetes dağıtım sürümleri Pod şablonunu (imaj, çevre değişkeni vb.) kapsar; veritabanındaki kalıcı verileri geriye sarmaz.

4.9 Deney 9: Çalışmanın Sonlandırılması ve Depolama Yönetimi (Ek 2 Bölüm 15-16)

Amaç: Uygulama iş yükleri kaldırıldığında kalıcı depolamanın (PVC ve PV) durumunu denetlemek, 'Delete' ve 'Retain' geri kazanım politikalarının farkını ve kalıcı depolama ile veri yedekleme arasındaki ayrımı ortaya koymak.

Uygulanan Komutlar ve Süreç: Dağıtımlar, servisler, kontrol Pod'u, ConfigMap ve Secret nesneleri 'kubectl delete' ile kaldırılmış, ancak 'db-data' PVC nesnesi bilinçli olarak korunmuştur:

Terminal / Kod: Deney 9 Kaynak Temizliği ve PVC Koruma Çıktısı

```
$ kubectl delete deployment web db -n hafta6
$ kubectl delete service web db -n hafta6
$ kubectl delete pod check -n hafta6
$ kubectl delete configmap app-config -n hafta6
$ kubectl delete secret app-secrets -n hafta6

$ kubectl get deployments,pods,services,configmaps,secrets -n hafta6
NAME                         DATA   AGE
configmap/kube-root-ca.crt   1      6m50s

$ kubectl get pvc db-data -n hafta6 -o wide
NAME      STATUS   VOLUME                                     CAPACITY   ACCESS MODES
STORAGECLASS   AGE    VOLUMEMODE
db-data   Bound    pvc-fff7d32f-6b0a-478c-9943-f274a6f5ed86   1Gi        RWO
standard       6m5s   Filesystem

$ kubectl get pv pvc-fff7d32f-6b0a-478c-9943-f274a6f5ed86
NAME                                       CAPACITY   ACCESS MODES   RECLAIM POLICY
STATUS   CLAIM            STORAGECLASS   AGE
pvc-fff7d32f-6b0a-478c-9943-f274a6f5ed86   1Gi        RWO            Delete
Bound    hafta6/db-data   standard       6m5s
```

Gözlemlenen Sonuç ve Değerlendirme: Uygulamanın tüm aktif iş yükleri sonlandırılmış olmasına rağmen, 'db-data' PVC'si ve diskteki fiziksel PV nesnesi 'Bound' durumunu korumuştur. Yapılandırma ve veritabanı Deployment'ı tekrar uygulandığında sistem aynı PVC'yi kullanarak eski verilere anında erişebilmektedir. Reclaim Policy 'Delete' olduğundan, eğer 'kubectl delete pvc db-data' komutu verilseydi, Kubernetes otomatik olarak alttaki PV'yi ve fiziksel depolama alanını da silecekti. 'Retain' politikasında ise PVC silinse dahi PV 'Released' durumuna geçer ve veriler disk üzerinde manuel müdahaleye kadar korunur.

5. BULUT ORTAMINA GEÇİŞ VE BÜTÜNLEŞİK DEĞERLENDİRME (EK 3 KAPSAMI)

Yerel kümede (Minikube / Docker Desktop) doğrulanan bir uygulamanın büyük bulut sağlayıcılarının (AWS EKS, Microsoft Azure AKS, Google Cloud GKE) yönetilen Kubernetes ortamlarına taşınması, mimari ve operasyonel açıdan belirli uyarlamalar gerektirir. Kılavuzun Ek 3 bölümünde ele alınan bu gereksinimler aşağıda karşılaştırmalı olarak analiz edilmiştir:

Mimari / Operasyonel Alan	Yerel Kubernetes Ortamı (Minikube)	Yönetilen Bulut Kubernetes (EKS / AKS / GKE)
Kontrol Düzlemi (Control Plane)	Geliştiricinin yerel makinesinde tek VM/konteyner olarak çalışır, yüksek erişilebilirlik (HA) yoktur.	Bulut sağlayıcı tarafından çoklu kullanılabilirlik alanında (Multi-AZ) SLA garantisiyle yönetilir.
İmaj Dağıtımı (Registry)	Yerel Docker daemon veya minikube image build ile yerel düğümlere yüklenir.	AWS ECR, Azure ACR veya GCP Artifact Registry gibi özel depolara gönderilir; IAM veya imagePullSecrets gerekir.
Dış Trafik Erişimi	ClusterIP servisleri dışa kapalıdır; geçici 'kubectl port-forward' ile test edilir.	Service type: LoadBalancer (AWS NLB, Azure LB) veya Cloud Ingress Controller (ALB, NGINX) ile gerçek IP sağlanır.
Kalıcı Depolama (Storage)	Yerel ana makine dizini (hostpath) kullanılır; düğüm arızasında veri kaybı riski vardır.	Bulut blok depolama sınıfları (AWS EBS gp3, Azure Disk, GCP Persistent Disk) veya paylaşımlı NFS (EFS) kullanılır.
Güvenlik ve Kimlik	Basit kubeconfig ve yerel servis hesapları kullanılır.	Bulut IAM rolleriyle entegre Pod Kimliği (AWS IRSA, Azure Workload Identity, GCP Workload Identity) kullanılır.
Maliyet Modeli	Donanım maliyeti hariç tamamen	Yönetilen küme saati, çalışan

	ücretsizdir.	EC2/VM düğümleri, yük dengeleyiciler ve ayrılan diskler için kullandıkça öde faturalandırması yapılır.

5.1 Paylaşımlı Sorumluluk Modeli ve Maliyet Yönetimi

Yönetilen bir Kubernetes hizmetinde (EKS/AKS/GKE) kontrol düzleminin sağlayıcı tarafından işletilmesi, uygulamanın tüm sorumluluğunun sağlayıcıya geçtiği anlamına gelmez. Bulut 'Paylaşımlı Sorumluluk Modeli'ne (Shared Responsibility Model) göre:

- **Sağlayıcının Sorumluluğu:** Kontrol düzleminin (API Server, etcd vb.) yüksek erişilebilirliği, çekirdek yamaları, veri merkezi güvenliği ve fiziksel donanım sağlayıcının sorumluluğundadır.
- **Müşterinin Sorumluluğu:** Uygulama kodunun güvenliği, doğru yapılandırma (ConfigMap/Secret), kaynak sınırlarının (requests/limits) belirlenmesi, sağlık denetimleri (probes), Pod güvenlik standartları, veri tabanı yedekleme stratejisi ve küme içi ağ politikaları tamamen müşterinin (öğrencinin/mühendisin) sorumluluğundadır.

> **[WARN] Maliyet Tuzağı: Deployment Silmek Maliyeti Bitirmez**
> Bulut denemeleri tamamlandığında 'kubectl delete deployment web db' komutunu çalıştırmak bulut maliyetlerini sonlandırmaz! Çünkü Deployment silindiğinde yalnızca Pod'lar kaldırılır. Arkada tahsis edilen Bulut Diskleri (AWS EBS / Azure Disk PersistentVolume), Statik IP adresleri, Düğüm Havuzundaki sanal sunucular (EC2 düğümleri) ve özellikle 'LoadBalancer' servisinin açtığı harici Yük Dengeleyici hizmeti arka planda çalışmaya ve saatlik ücret üretmeye devam eder. Tüm harcamaları sonlandırmak için ilişkili PVC'lerin, Service nesnelerinin ve nihayetinde kümenin kendisinin ('eksctl delete cluster' / 'az aks delete') tamamen imha edilmesi gerekmektedir.

6. KAVRAMSAL AÇIKLAMALAR VE ÇALIŞMA SORULARI

Ders notunun Bölüm 7.3 ve Kılavuzun Ek 2 Bölüm 17 başlıklarında yer alan altı temel kavramsal soru, gerçekleştirilen kontrollü deneylerin sonuçları ve Kubernetes mimarisi temelinde kapsamlı bir biçimde yanıtlanmıştır:

Soru 1: Web uygulamasına erişim ile web bileşeninin PostgreSQL'e erişimi nasıl sağlanır?

Yanıt: Kubernetes ortamında Pod'lar geçici (ephemeral) varlıklardır; her yeniden başlatmada farklı bir dinamik IP adresi alırlar. Bu nedenle bileşenler birbirine doğrudan Pod IP'leri üzerinden bağlanamaz. İletişim iki farklı 'Service' nesnesi ve Kubernetes iç ağ mekanizmalarıyla çözülmüştür:

- **Web Bileşeninin Veritabanına Erişimi:** Web bileşeni, veritabanına '04-services.yaml' dosyasında tanımlanan 'service/db' nesnesi üzerinden erişir. Bu servis 'ClusterIP' türündedir ve küme içinde sabit bir sanal IP (10.111.73.51) ile 'db' adında bir DNS kaydı üretir. Web konteynerine ConfigMap üzerinden 'DB_HOST=db' ve 'DB_PORT=5432' verilmiştir. Web Pod'u istek gönderdiğinde CoreDNS bu ismi sanal IP'ye, kube-proxy ise bu IP'yi arkadaki PostgreSQL Pod'unun (10.244.0.3) 5432 portuna iletir. Veritabanı doğrudan internete açılmamış, yalnızca küme içi güvenli ağda tutulmuştur.
- **Kullanıcıların Web Uygulamasına Erişimi:** Kullanıcıların web arayüzüne erişimi üretim ortamında 'LoadBalancer' veya 'Ingress Controller' ile sağlanırken, bu yerel laboratuvar çalışmasında 'service/web' (ClusterIP) kullanılmıştır. Geliştiricinin yerel makinesinden bu servise ulaşabilmesi için 'kubectl port-forward -n hafta6 service/web 8080:8000' komutuyla bir tünel açılmıştır. Bu tünel, yerel makinenin 127.0.0.1:8080 portuna gelen istekleri küme içindeki 'service/web' üzerinden EndpointSlice'ta 'ready: true' olan web Pod'unun 8000 portuna yönlendirmiştir.

Soru 2: Silinen web Pod'unun yerine neden yeni Pod oluşturulur? Konteyner yeniden başlatılması ile Pod yeniden oluşturulması arasındaki fark nedir?

Yanıt: Web Pod'u 'Deployment' nesnesi ve onun arka planda yönettiği 'ReplicaSet' denetleyicisi tarafından idare edilmektedir. Dağıtım tanımında 'spec.replicas: 1' beyan edilmiştir. Kubernetes kontrol döngüsü (reconciliation loop), kümedeki mevcut Pod sayısını sürekli gözlemler. Bir web Pod'u silindiğinde aktif kopya sayısı 0'a düşer. ReplicaSet istenen durum (1) ile gerçek durum (0) arasındaki farkı kapatmak için derhal yeni bir Pod planlar (Self-Healing).

Konteynerin Yeniden Başlatılması ile Pod'un Yeniden Oluşturulması Arasındaki Farklar:

Karşılaştırma Kriteri	Konteynerin Yeniden Başlatılması (Container Restart)	Pod'un Yeniden Oluşturulması (Pod Re-creation)
Tetikleyen Mekanizma	Kubelet (Liveness Probe başarısızlığı veya sürecin çökmesi).	ReplicaSet / Deployment denetleyicisi (Pod silinmesi veya düğüm kaybı).

Pod Kimliği (UID) ve IP	Pod UID'si ve Pod IP adresi DEĞİŞMEZ, tamamen aynı kalır.	Eski Pod tamamen yok edilir; YENİ bir UID ve YENİ bir IP adresi atanır.
Konteyner Yaşam Döngüsü	Pod kabuğu yerinde kalır; yalnızca içindeki konteyner süreci yeniden başlatılır.	Tüm konteynerler sıfırdan oluşturulur, imaj yeniden çekilebilir.
Dosya Sistemi Etkisi	Konteynerin yazılabilir katmanı sıfırlanır (örn. /tmp/live-fail yok olur).	Pod düzeyindeki boşluklar (emptyDir) ve geçici katmanlar tamamen silinir.
Gösterge Belirtisi	kubectl get pod çıktısında RESTARTS sayacı 1 artar.	RESTARTS 0 olur, Pod adı ve UID'si değişir, AGE süresi sıfırlanır.

Soru 3: Kayıtlar nerede saklanır? PostgreSQL Pod'u yeniden oluşturulduğunda neden korunur?

Yanıt: Konteynerlerin yerel dosya sistemi geçicidir (ephemeral layer). Bir konteyner silindiğinde bu katmandaki tüm veriler kaybolur. Ancak PostgreSQL dağıtımımızda veriler konteynerin içine değil; Kubernetes'in 'PersistentVolumeClaim (db-data)' nesnesi aracılığıyla bağlanan harici bir 'PersistentVolume' (PV) üzerine yazılmaktadır.

Çalışma dosyamızda (03-db.yaml) '/var/lib/postgresql/data' dizini 'data' adlı birime, bu birim de 'db-data' PVC'sine bağlanmıştır. Ayrıca 'PGDATA' değişkeni '/var/lib/postgresql/data/pgdata' yapılarak veritabanı dosyalarının bu kalıcı alt dizinde tutulması sağlanmıştır. PostgreSQL Pod'u silindiğinde Pod'un kendisi ölür; fakat bağımsız bir Kubernetes nesnesi olan 'db-data' PVC ve diskteki PV varlığını sürdürür. Yeni db Pod'u ayağa kalktığında Deployment aynı PVC'yi yeni Pod'a bağlar. PostgreSQL başlatma betiği PGDATA dizininde mevcut veritabanı kümelerini (cluster data) bulur, yeni bir kurulum yapmaz ve mevcut veritabanını doğrudan kullanıma açar. Bu sayede Kalıcılık A ve B kayıtları sıfır kayıpla korunmuştur.

Soru 4: Hazır olma kontrolünün başarısızlığı ile canlılık kontrolünün başarısızlığı uygulamayı nasıl etkiler?

Yanıt: Kubernetes mimarisinde Hazır Olma Kontrolü (Readiness Probe) ile Canlılık Kontrolü (Liveness Probe) tamamen farklı amaçlara hizmet eder ve başarısızlıklarında zıt tepkiler üretir:

- **Hazır Olma Kontrolü (Readiness Probe):** Konteynerin kullanıcı isteklerini kabul etmeye hazır olup olmadığını denetler (örneğin veritabanı bağlantısı sağlıklı mı, önbellekler yüklendi mi?). Başarısız Olduğunda: Kubelet konteyneri ASLA öldürmez veya yeniden

başlatmaz! Bunun yerine Pod'un durumunu 'READY: 0/1' yapar ve EndpointSlice/Service denetleyicisine haber verir. Servis, bu Pod'u trafik yönlendirme havuzundan çıkarır ('ready: false'). Kullanıcılar hata almaz, trafik varsa diğer hazır Pod'lara akar. Sorun düzeldiğinde (örn. veritabanı geri geldiğinde) Pod otomatik olarak tekrar havuza alınır.

- **Canlılık Kontrolü (Liveness Probe):** Konteyner sürecinin hayatta olup olmadığını, çıkmaza (deadlock) girip girmediğini denetler. Başarısız Olduğunda: Kubelet, uygulamanın kendini kurtaramayacağına hükmeder ve konteyneri zorla sonlandırarak (SIGKILL) YENİDEN BAŞLATIR (restart). Deney 6'da görüldüğü üzere RESTARTS sayısı artar. Eğer veritabanı kesintisinde liveness probe veritabanını denetleyecek şekilde yanlış yapılandırılsaydı, veritabanı kapalı kaldığı sürece web konteyneri sonsuz bir yeniden başlatma döngüsüne (CrashLoopBackOff) girer ve gereksiz kaynak tüketirdi.

Soru 5: Web kopya sayısı değiştiğinde hangi bileşenler değişir? PostgreSQL neden aynı şekilde çoğaltılmaz?

Yanıt: Web kopya sayısı 1'den 2'ye çıkarıldığında kümede şu bileşenler değişir:
1. Deployment denetleyicisi ReplicaSet'in hedef kopya sayısını 2 yapar.
2. Kubernetes Scheduler, ikinci web Pod'u için kaynak kontrolü yapar (100m CPU, 128Mi RAM) ve Pod'u uygun düğüme atar.
3. Kubelet ikinci web konteynerini başlatır; başlangıç ve hazır olma kontrolleri tamamlandığında Pod 'Running 1/1' olur.
4. En kritik değişiklik: 'EndpointSlice' nesnesine ikinci Pod'un IP adresi eklenir ve 'ready: true' yapılır. Kube-proxy yönlendirme tablolarını güncelleyerek gelen HTTP isteklerini bu iki Pod arasında paylaştırmaya başlar.

PostgreSQL'in Neden Aynı Şekilde Çoğaltılamayacağı: Web servisi tamamen durumsuzdur (stateless); oturum bilgisi tutmaz ve gelen isteği veritabanından okuyup döner. Bu nedenle kopya sayısı yatayda rahatça artırılabilir. Ancak PostgreSQL ilişkisel ve durumsal (stateful) bir veritabanıdır. Kılavuzdaki kalıcı depolama 'ReadWriteOnce' modundadır; yani aynı anda yalnızca tek bir düğüm tarafından bağlanabilir. Eğer PostgreSQL Deployment kopyası 2 yapılırsa, iki farklı Pod aynı veri dizini (PGDATA) üzerinde eşzamanlı işlem kilitleri (row locks, table locks) almaya ve veri dosyalarını değiştirmeye çalışır. Bu durum ACID garantisini bozar, yarış durumlarına (race condition) yol açar ve veritabanı tablolarının fiziksel olarak bozulmasına (data corruption) neden olur. Veritabanını ölçeklemek için Deployment değil, birincil-ikincil (master-replica) replikasyon protokolleri çalıştıran 'StatefulSet' mimarileri veya bulut veritabanı servisleri (DBaaS) gereklidir.

Soru 6: Yeni sürüme geçiş ve önceki sürüme dönüş nasıl doğrulanır? Önceki uygulama sürümüne dönmek neden veritabanını da önceki hâline getirmez?

Yanıt: Yeni sürüme geçiş 'kubectl set image deployment/web web=dersnotlari-web:1.2' komutuyla tetiklenir. Doğrulama adımları:
1. 'kubectl rollout status' komutuyla aşamalı güncellemenin (RollingUpdate) tamamlandığı izlenir.
2. 'kubectl get deployment web -o jsonpath={...image}' ile çalışan aktif imaj etiketinin 'dersnotlari-web:1.2' olduğu doğrulanır.
3. Web uygulamasının '/health/ready' uç noktasından 'version: 1.2' yanıtı alınır ve arayüzde 'Sürüm: 1.2' başlığı görülür.
Önceki sürüme dönüş ise 'kubectl rollout undo deployment/web' komutuyla gerçekleştirilir; aktif imajın tekrar 'dersnotlari-web:1.1' olduğu ve /health/ready'nin 'version: 1.1' döndürdüğü doğrulanır.

Geri Dönüşün Veritabanını Neden Geri Getirmediği: Kubernetes 'rollout undo' komutu, yalnızca Deployment manifestosundaki Pod Şablonunu (Pod Template - imaj sürümü, ortam değişkenleri, etiketler vb.) önceki revizyona geri çeker. Veritabanı ve kalıcı birimler (PV/PVC) Deployment yaşam döngüsünden tamamen bağımsız nesnelerdir. Uygulama sürümü 1.2 iken kullanıcıların kaydettiği veriler, fiziksel olarak PostgreSQL'in disk tablolarına yazılmıştır. Kubernetes'in veritabanı işlemlerini geriye alma (rollback database transactions) veya disk bloklarını geçmişe sarma gibi bir yetkisi ve mekanizması yoktur. Uygulama kodunu eski sürüme almak, veritabanını da eski haline getirmez. Veritabanı durumunu geri almak için periyodik veritabanı yedeklerinin (pg_dump, snapshot) veya geriye dönük veritabanı şema migrasyonlarının (database migration rollbacks) kullanılması şarttır.

7. ÇALIŞMA KODLARI VE MANİFEST DOSYALARI

Çalışma kapsamında hazırlanan, yerel Kubernetes kümesinde test edilip doğrulanan tüm kaynak kodlar ve bildirimsel YAML manifestoları aşağıda eksiksiz olarak belgelenmiştir:

7.1 Flask Web Uygulaması Kaynak Kodu (app.py)

Terminal / Kod: app.py

```
import os
from pathlib import Path
from flask import Flask, render_template, request, redirect, url_for, jsonify
import psycopg

APP_VERSION = "1.1"
```

```
app = Flask(__name__)

# Secret key configuration
app_secret_file = os.environ.get("APP_SECRET_FILE", "/run/secrets/app_secret")
if Path(app_secret_file).exists():
    with open(app_secret_file, "r") as f:
        app.secret_key = f.read().strip()
else:
    app.secret_key = os.environ.get("APP_SECRET", "default-dev-secret-key")

# Database configuration
db_host = os.environ.get("DB_HOST", "db")
db_port = int(os.environ.get("DB_PORT", "5432"))
db_name = os.environ.get("DB_NAME", "dersnotlari")
db_user = os.environ.get("DB_USER", "dersuser")

db_password_file = os.environ.get("DB_PASSWORD_FILE", "/run/secrets/db_password")
if Path(db_password_file).exists():
    with open(db_password_file, "r") as f:
        db_password = f.read().strip()
else:
    db_password = os.environ.get("DB_PASSWORD", "postgrespassword")

def get_db_connection():
    return psycopg.connect(
        host=db_host,
        port=db_port,
        dbname=db_name,
        user=db_user,
        password=db_password,
        connect_timeout=3
    )

def init_db():
    try:
        with get_db_connection() as conn:
            with conn.cursor() as cur:
                cur.execute("""
                    CREATE TABLE IF NOT EXISTS dersnotlari (
                        id SERIAL PRIMARY KEY,
                        numara VARCHAR(50) NOT NULL,
                        baslik VARCHAR(200) NOT NULL,
                        aciklama TEXT NOT NULL,
                        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
                    );
                """)
```

```
                conn.commit()
    except Exception as e:
        print(f"Database init warning (might be offline yet): {e}")

@app.route("/", methods=["GET", "POST"])
def index():
    if request.method == "POST":
        numara = request.form.get("numara", "").strip()
        baslik = request.form.get("baslik", "").strip()
        aciklama = request.form.get("aciklama", "").strip()
        if numara and baslik:
            try:
                with get_db_connection() as conn:
                    with conn.cursor() as cur:
                        cur.execute(
                            "INSERT INTO dersnotlari (numara, baslik, aciklama) VALUES
(%s, %s, %s)",
                            (numara, baslik, aciklama)
                        )
                        conn.commit()
            except Exception as e:
                return f"Veritabanı kayıt hatası: {e}", 500
        return redirect(url_for("index"))

    # GET
    kayitlar = []
    db_error = None
    try:
        with get_db_connection() as conn:
            with conn.cursor() as cur:
                cur.execute("SELECT numara, baslik, aciklama, created_at FROM
dersnotlari ORDER BY id ASC")
                kayitlar = cur.fetchall()
    except Exception as e:
        db_error = str(e)

    return render_template("index.html", kayitlar=kayitlar, version=APP_VERSION,
db_error=db_error)

@app.route("/health/live")
def health_live():
    return jsonify({"status": "ok"}), 200

@app.route("/health/ready")
def health_ready():
    try:
        with get_db_connection() as conn:
```

```
            with conn.cursor() as cur:
                cur.execute("SELECT 1;")
                cur.fetchone()
        return jsonify({"status": "ready", "version": APP_VERSION}), 200
    except Exception as e:
        return jsonify({"status": "unready", "error": str(e), "version": APP_VERSION}),
503

if __name__ == "__main__":
    init_db()
    app.run(host="0.0.0.0", port=8000)
```

7.2 Web Arayüzü HTML Şablonu (templates/index.html)

Terminal / Kod: templates/index.html

```
<!DOCTYPE html>
<html lang="tr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Ders Notları Kayıt Uygulaması - v{{ version }}</title>
    <style>
        body {
            font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto,
Helvetica, Arial, sans-serif;
            background-color: #f4f6f9;
            margin: 0;
            padding: 30px;
            color: #333;
        }
        .container {
            max-width: 800px;
            margin: 0 auto;
            background: #fff;
            padding: 30px;
            border-radius: 8px;
            box-shadow: 0 4px 12px rgba(0,0,0,0.08);
        }
        .header {
            display: flex;
            justify-content: space-between;
            align-items: center;
            border-bottom: 2px solid #eaedf1;
            padding-bottom: 15px;
            margin-bottom: 25px;
```

```
        }
        .header h1 {
            margin: 0;
            font-size: 24px;
            color: #1a202c;
        }
        .badge {
            background-color: #2b6cb0;
            color: white;
            padding: 6px 14px;
            border-radius: 20px;
            font-size: 14px;
            font-weight: 600;
        }
        .form-group {
            margin-bottom: 15px;
        }
        label {
            display: block;
            margin-bottom: 6px;
            font-weight: 500;
            color: #4a5568;
        }
        input[type="text"], textarea {
            width: 100%;
            padding: 10px;
            border: 1px solid #cbd5e0;
            border-radius: 6px;
            box-sizing: border-box;
            font-size: 14px;
        }
        button {
            background-color: #3182ce;
            color: white;
            padding: 10px 20px;
            border: none;
            border-radius: 6px;
            font-size: 15px;
            font-weight: 500;
            cursor: pointer;
            transition: background 0.2s;
        }
        button:hover {
            background-color: #2b6cb0;
        }
        table {
            width: 100%;
```

```
            border-collapse: collapse;
            margin-top: 30px;
        }
        th, td {
            text-align: left;
            padding: 12px 14px;
            border-bottom: 1px solid #e2e8f0;
        }
        th {
            background-color: #f7fafc;
            color: #4a5568;
            font-weight: 600;
        }
        .alert-error {
            background-color: #fed7d7;
            color: #9b2c2c;
            padding: 12px;
            border-radius: 6px;
            margin-bottom: 20px;
        }
    </style>
</head>
<body>
    <div class="container">
        <div class="header">
            <h1>Ders Notları Kayıt Uygulaması</h1>
            <span class="badge">Sürüm: {{ version }}</span>
        </div>

        {% if db_error %}
        <div class="alert-error">
            <strong>Veritabanı Bağlantı Hatası:</strong> {{ db_error }}
        </div>
        {% endif %}

        <form method="POST" action="/">
            <div class="form-group">
                <label for="numara">Kayıt Numarası:</label>
                <input type="text" id="numara" name="numara" placeholder="Örn: 101"
required>
            </div>
            <div class="form-group">
                <label for="baslik">Not Başlığı:</label>
                <input type="text" id="baslik" name="baslik" placeholder="Örn: Kalıcılık
A" required>
            </div>
            <div class="form-group">
```

```
                <label for="aciklama">Açıklama:</label>
                <textarea id="aciklama" name="aciklama" rows="3" placeholder="Not
içeriği ve açıklaması..."></textarea>
            </div>
            <button type="submit">Kayıt Ekle</button>
        </form>

        <h2>Kayıtlı Ders Notları</h2>
        <table>
            <thead>
                <tr>
                    <th>No</th>
                    <th>Başlık</th>
                    <th>Açıklama</th>
                    <th>Tarih</th>
                </tr>
            </thead>
            <tbody>
                {% for kayit in kayitlar %}
                <tr>
                    <td><strong>{{ kayit[0] }}</strong></td>
                    <td>{{ kayit[1] }}</td>
                    <td>{{ kayit[2] }}</td>
                    <td><small>{{ kayit[3] }}</small></td>
                </tr>
                {% else %}
                <tr>
                    <td colspan="4" style="text-align:center; color:#a0aec0;
padding:20px;">Henüz kayıt bulunmamaktadır.</td>
                </tr>
                {% endfor %}
            </tbody>
        </table>
    </div>
</body>
</html>
```

7.3 Docker Yapılandırması (Dockerfile ve .dockerignore)

Terminal / Kod: Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["python", "app.py"]
```

Terminal / Kod: .dockerignore
```
__pycache__/
*.pyc
*.pyo
*.pyd
.env
secrets/
k8s/
*.docx
*.pdf
ek*.txt
hafta*.txt
```

7.4 Kubernetes Manifestoları (k8s Klasörü)

Terminal / Kod: 01-config.yaml
```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
  namespace: hafta6
data:
  DB_HOST: db
  DB_PORT: "5432"
  DB_NAME: dersnotlari
  DB_USER: dersuser
  COOKIE_SECURE: "false"
```

Terminal / Kod: 02-pvc.yaml
```
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: db-data
```

```
  namespace: hafta6
spec:
  accessModes:
    - ReadWriteOnce
  resources:
    requests:
      storage: 1Gi
```

Terminal / Kod: 03-db.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: db
  namespace: hafta6
spec:
  replicas: 1
  strategy:
    type: Recreate
  selector:
    matchLabels:
      app: db
  template:
    metadata:
      labels:
        app: db
    spec:
      automountServiceAccountToken: false
      containers:
      - name: db
        image: postgres:16
        env:
        - name: POSTGRES_DB
          value: dersnotlari
        - name: POSTGRES_USER
          value: dersuser
        - name: POSTGRES_PASSWORD_FILE
          value: /run/secrets/db_password
        - name: PGDATA
          value: /var/lib/postgresql/data/pgdata
        ports:
        - containerPort: 5432
        readinessProbe:
          exec:
            command: [pg_isready, -U, dersuser, -d, dersnotlari]
          initialDelaySeconds: 5
          periodSeconds: 5
```

```
        resources:
          requests: {cpu: 100m, memory: 128Mi}
          limits: {cpu: 500m, memory: 512Mi}
        volumeMounts:
        - name: data
          mountPath: /var/lib/postgresql/data
        - name: secret
          mountPath: /run/secrets
          readOnly: true
      volumes:
      - name: data
        persistentVolumeClaim:
          claimName: db-data
      - name: secret
        secret:
          secretName: app-secrets
          items:
          - key: db_password
            path: db_password
```

Terminal / Kod: 04-services.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: db
  namespace: hafta6
spec:
  type: ClusterIP
  selector:
    app: db
  ports:
  - port: 5432
    targetPort: 5432
---
apiVersion: v1
kind: Service
metadata:
  name: web
  namespace: hafta6
spec:
  type: ClusterIP
  selector:
    app: web
  ports:
  - port: 8000
```

```
    targetPort: 8000
```

Terminal / Kod: 05-web.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: web
  namespace: hafta6
spec:
  replicas: 1
  revisionHistoryLimit: 5
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 0
      maxSurge: 1
  selector:
    matchLabels:
      app: web
  template:
    metadata:
      labels:
        app: web
    spec:
      automountServiceAccountToken: false
      containers:
      - name: web
        image: dersnotlari-web:1.1
        imagePullPolicy: Never
        ports:
        - containerPort: 8000
        envFrom:
        - configMapRef:
            name: app-config
        env:
        - name: DB_PASSWORD_FILE
          value: /run/secrets/db_password
        - name: APP_SECRET_FILE
          value: /run/secrets/app_secret
        startupProbe:
          httpGet:
            path: /health/live
            port: 8000
          periodSeconds: 5
          failureThreshold: 30
        readinessProbe:
```

```
          httpGet:
            path: /health/ready
            port: 8000
          periodSeconds: 5
          timeoutSeconds: 10
          failureThreshold: 2
        livenessProbe:
          exec:
            command:
            - python
            - -c
            - >
              import pathlib, urllib.request;
              assert not pathlib.Path('/tmp/live-fail').exists();
              urllib.request.urlopen(
              'http://127.0.0.1:8000/health/live', timeout=2).read()
          periodSeconds: 5
          timeoutSeconds: 3
          failureThreshold: 3
        resources:
          requests: {cpu: 100m, memory: 128Mi}
          limits: {cpu: 500m, memory: 256Mi}
        volumeMounts:
        - name: secret
          mountPath: /run/secrets
          readOnly: true
      volumes:
      - name: secret
        secret:
          secretName: app-secrets
```

Terminal / Kod: 06-check.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: check
  namespace: hafta6
spec:
  restartPolicy: Never
  containers:
  - name: check
    image: dersnotlari-web:1.1
    imagePullPolicy: Never
    command: [python, -c, "import time; time.sleep(86400)"]
    resources:
      requests: {cpu: 10m, memory: 32Mi}
```

```
      limits: {cpu: 100m, memory: 64Mi}
```

8. KAYNAKÇA

[1] National Institute of Standards and Technology (NIST), 'The NIST Definition of Cloud Computing', Special Publication 800-145, Sept. 2011.

[2] Kubernetes Documentation, 'Kubernetes Concepts and Architecture', https://kubernetes.io/docs/concepts/ (Erişim: Kasım 2026).

[3] Docker Documentation, 'Docker Desktop Kubernetes Environment', https://docs.docker.com/desktop/use-desktop/kubernetes/ (Erişim: Kasım 2026).

[4] Minikube Documentation, 'Minikube Handbook and Driver Architecture', https://minikube.sigs.k8s.io/docs/ (Erişim: Kasım 2026).

[5] Kubernetes Documentation, 'Configure Liveness, Readiness and Startup Probes', https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/.

[6] Kubernetes Documentation, 'Persistent Volumes and Storage Classes', https://kubernetes.io/docs/concepts/storage/persistent-volumes/.

[7] Kubernetes Documentation, 'Services, Load Balancing, and Networking', https://kubernetes.io/docs/concepts/services-networking/service/.

[8] Kubernetes Documentation, 'Managing Resources for Containers', https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/.

[9] Kubernetes Documentation, 'Deployments and Rolling Updates', https://kubernetes.io/docs/concepts/workloads/controllers/deployment/.

[10] Cloud Native Computing Foundation (CNCF), 'Cloud Native Landscape and Container Orchestration Patterns', 2026.